
B28: SQL SuperHero Program : Session 30

1 message

Vishal Jaiswal <vishaldbdev@gmail.com>
Bcc: satishhattekar1997@gmail.com

6 April 2025 at 20:54

Can I write a return statement in a function before DML? What impact in such case

Yes, There is a basic concept that yes we can write DML as we see it in previous examples. But never forget that basic nature of function is to return something. so, as soon as it find **RETURN** clause, it will not go and check further lines.

```
create or replace function upd_sal(p_emp_id number) return number as
l_salary employee.salary%type;
Begin
  select salary into l_salary from employee where employee_id = p_emp_id;
  return l_salary;
  update employee set salary = salary + 500 where employee_id = p_emp_id;

End;
```

Function created.

```
select upd_sal(100) from dual
```

UPD_SAL(100)
25500

Handwritten notes:
A blue arrow points from the text "DML statement" to the update statement in the function code.
A blue arrow points from the text "function called in select here" to the select statement in the query.

Can I write a DDL Inside Function

```
create or replace function ddl_function return number as
Begin
  Execute Immediate 'create table xxx(id number)';
  Return 1;
End;
```

Syntax *name* *mandatory* *data type of output*

```

create or replace function ddl_function return number as
Begin
    Execute Immediate 'create table xxx(id number)';
    Return 1;
End;

```

DDL Statement

Function created. *→ yes, we can create*

```

select ddl_function() from dual

```

→ can not call in select stmt

ORA-14552: cannot perform a DDL, commit or rollback inside a query or DML

More Details: <https://docs.oracle.com/error-help/db/ora-14552>

```

declare
    a number;
Begin
    a := ddl_function();
End;

```

} - call here

Statement processed.

```

select * from user_tables

```

Table created

TABLE_NAME	TABLESPACE_NAME	CLUSTER_NAME	IOT_NAME	STATUS
XXX	LIVESQL_USERS	-	-	VALID

--> Understand the basic concept of function is to : compute something. If we are doing anything different (calling dml, calling ddl, calling procedure in it), then we can not call this function in the select statement.

Can a Function return case?

YES. case return 1 value and function returns 1 value. so yes.

```

Create or replace function is_region_exists (p_id number, p_name varchar2) return varchar as
l_count number;
l_exists Varchar2(10);
Begin
    select count(*) into l_count from hr.regions where region_id = p_id and region_name = p_name;

    Return case when l_count > 0 then 'YES' else 'NO' end;
End;

```

Function created.

```
select is_region_exists(1, 'Americas') from dual
```

IS_REGION_EXISTS(1, 'AMERICAS')
NO

→ case return 1 value &
function return 1 value
↓
yes, we can call
case in return

Procedure

Procedures can be used in the same way like functions.
We can write multiple DML operations and all will be executed together.

create or replace procedure insert_data(p_id number, p_name varchar2) As

Begin

Insert Into Regions Values (p_id,p_name);

End insert_data;

```

create or replace procedure insert_data(p_id number, p_name varchar2) as
Begin
insert into regions values (p_id, p_name);
End;

Procedure created.

```

Syntax → Name → Data type of i/p → i/p Parameter → no return

The above procedure is written to insert data in regions table. It takes 2 input. First a number and another a string and accordingly insert data in regions tables. There are 4 ways we can call a procedure :

--> 1st way of calling a procedure

```
Begin  
  insert_data(5,'India');  
End;
```

--> 2nd way of calling Procedure

```
Exec insert_data(5,'India');
```

--> 3rd way of calling procedure

```
Execute insert_data(5,'India');
```

--> 4th way of calling procedure

```
Call insert_data(5,'India');
```

Once run this above procedure it will insert data in the table.

The screenshot shows a SQL Developer interface. The top pane displays the execution of a PL/SQL procedure:

```
Begin  
  insert_data(5,'India');  
End;
```

Below the code, it says "Statement processed." To the right of this code, a red handwritten note says "] - procedure called" with a green arrow pointing from the procedure call to the table query result below.

The bottom pane shows the execution of a query:

```
select * from regions
```

Below the query, a table is displayed with 5 rows:

REGION_ID	REGION_NAME
1	Europe
2	Americas
3	Asia
4	Middle East and Africa
5	India

Below the table, it says "5 rows selected." To the right of the table, a green handwritten note says "Data inserted" with a green arrow pointing from the note to the row containing "India".

Let me write the same procedure with 2 statement, one insert and one delete

create or replace procedure insert_data(p_id number, p_name varchar2, p_id1 number) As

```
Begin  
  Insert Into Regions Values (p_id,p_name);  
  delete from regions where region_id = p_id1;
```

```
End insert_data;
```

This will insert
This will delete

```

create or replace procedure insert_data(p_id number, p_name varchar2, p_id1 number) as
Begin
insert into regions values (p_id, p_name);
delete from regions where region_id = p_id1;
End;

Procedure created.

execute insert_data(6,'London',5)

Statement processed.

```

In the above code, we are passing 3 parameters. for the first 2 parameters, it will insert data in regions table and for the 3rd parameter, it will delete the data. lets test :

Execute `insert_data(6,'London', 5);`

--> When we call this procedure with the above 3 parameters, it will insert data for 6 as regions_id and London as region_name and also delete record from table for region_id = 5

```

execute insert_data(6,'London',5)

Statement processed.

select * from regions

```

this deleted

REGION_ID	REGION_NAME
1	Europe
2	Americas
3	Asia
4	Middle East and Africa
6	London

inserted

5 rows selected.

--> See the output. 6 as region_id inserted and 5 as region_id deleted.

=====

Can I write a Function within a Procedure?

Yes, Whatever we want we can do. Procedure is nothing but a wrapper, which let us know that these sql statements are stored in database object and when we called these are executed. we do not have to write it again and again. These are a part of reusable program unit.

Lets Create a function first and then call it within a procedure

```
create or replace function get_salary(p_id number) return number as
l_salary number;
Begin
    select salary into l_salary from hr.employees where employee_id = p_id;
    return l_salary;
End;
```

```
create or replace function get_salary(p_id number) return number as
    l_sal number;
Begin
    select salary into l_sal from hr.employees where employee_id = p_id;
    return l_sal;
End;
```

Function created.

Handwritten: } - fn created

```
select get_salary(100) from dual
```

Handwritten: → its working

GET_SALARY(100)
24000

Now Lets call it within a procedure.

```
create or replace procedure insert_data(p_id number, p_name varchar2, p_id1 number, p_emp_id number) as
l_salary number;
Begin
    delete from regions where region_id = p_id;
    insert into regions values (p_id1, p_name);
    update employees set salary = 100 where employee_id = p_emp_id;
    select get_salary(p_emp_id) into l_salary from dual;
    dbms_output.put_line(l_salary);
End;
```

Handwritten: → fn called here

Procedure created. *Handwritten: → procedure created*

```
execute insert_data(6,'India',5,100)
```

```
Statement processed.  
24000
```

→ Procedure executed

↓ print from function

=====
-> Same you can write a procedure in a function, but the limitation is , we can not call that function in a select statement.

```
create or replace function get_salary(p_id number) return number as  
l_salary number;  
Begin  
    select salary into l_salary from hr.employees where employee_id = p_id;  
    Begin  
        insert_data(2,'XYZ',4,110);  
    End;  
    return l_salary;  
  
End get_salary;
```

] - Procedure called here

```
Function created.
```

```
select get_salary(100) from dual
```

```
ORA-14551: cannot perform a DML operation inside a query
```

] → limitation

=====
If Function Or Procedure both have input parameters while creating then

1. Number of Argument and Type of Argument must be same
2. It will pass in same sequence as it was defined
3. We can pass by name and then sequence does not matter

```
Create or replace procedure insert_data(p_id1 number,p_id IN number, p_name varchar2) as  
Begin  
    -- p_id := 7;  
    insert into regions values (p_id, p_name);  
    -- Return;  
    delete from regions where region_id = p_id1;  
End;
```

If we call by default :

```
exec insert_data(2,7,'New Delhi');
```

The first Parameter 2 means p_id1 .

The 2nd Parameter 7 means p_id

The 3rd Parameter 'New Delhi' means p_name

If you want, you can call in any sequence by giving the name:

```
exec insert_data(p_name=>'New Delhi', p_id1 => 2, p_id => 7);
```

=====

can we write "RETURN" in a procedure?

YES. we can write. There is no issue in compilation , but when the pointer reaches "RETURN", it returns from there and does not execute anything after that.

create or replace procedure insert_data(p_id number, p_name varchar2, p_id1 number) As

Begin

Insert Into Regions Values (p_id,p_name);

RETURN;

delete from regions where region_id = p_id1;

End insert_data;

```
1 create or replace procedure insert_data (p_region_id1 number, p_region_id number, p_region_name varchar2) as
2   l_sal number;
3 Begin
4   insert into regions values (p_region_id, p_region_name);
5   return;
6   delete from regions where region_id = p_region_id1;
7
8 End;
```

→ Here we wrote
return.

✓ compiled

Now let me test this procedure again:

```
exec insert_data(2,7,'New delhi')
```

Statement processed.

--> When we call this procedure with the above 3 parameters, it will insert data for 7 as regions_id and "New Delhi" as region_name

but will not delete record from table for region_id = 2, because we wrote "RETURN". so in next line it found return and return from there


```
select * from regions
```

REGION_ID	REGION_NAME
1	Europe
2	Americas
3	Asia
4	Middle East and Africa
7	New delhi
6	London

→ Not deleted (Because we wrote return)

→ New inserted

6 rows selected.

Now, suppose if we comment out that RETURN CLAUSE

```
create or replace procedure insert_data (p_region_id1 number, p_region_id number, p_region_name varchar2) as
    l_sal number;
Begin
    insert into regions values (p_region_id, p_region_name);
    -- return;
    delete from regions where region_id = p_region_id1;

End;

Procedure created.
```

→ Comment out

Now, the delete will work and data will be deleted

```
exec insert_data(2,7,'New delhi')
```

Statement processed.

```
select * from regions
```

REGION_ID	REGION_NAME
1	Europe
3	Asia
4	Middle East and Africa
7	New delhi
7	New delhi
6	London

6 rows selected.

2 deleted
Now.

Main difference between IN and OUT parameters is that IN is used to pass data to the database via SQL query, while OUT parameter is used to read results from the database,

- **IN** : A variable passed in this mode is of read only nature. This is to say, the value cannot be changed and its scope is restricted within the procedure. The procedure receives a value from this argument when the procedure is called. The IN parameter can be referenced within the called program and the called program cannot assign a new value to the IN parameter. It is fixed.
- **OUT** : In this mode, a variable is write only and can be passed back to the calling program. It cannot be read inside the procedure and needs to be assigned a value. The called program can reference and assign new values to the formal parameter.
- **INOUT** : This procedure has features of both IN and OUT mode. The procedure can also read the variables value and can also change it to pass it to the calling function.

Function with OUT and IN OUT Parameter

YES, we can use but we can not call such function in a select statement. Don't mix OUT parameter with Return. Both are completely different

create or replace function test1(a in number, b in number, c out number) return number as

Begin

c:= a+b;

return a-b;

End Test1;

```

create or replace function test1(A IN number, b in number, c out number) return number as
Begin
    c := a+b;
    return a-b;
End test1;

Function created.

```

out parameter

Must return a value.

The only thing is, we can not call such functions in a select statement. Function can take out parameters, but can not be used in select statements.

only 2 parameter based

```

select test1(10,5) from dual

```

ORA-06553: PLS-306: wrong number or types of arguments in call to 'TEST1'

when created fn, we gave 3 parameter (2 in 1 out)

our function

```

Declare
    C number;
Begin
    select test1(10,5,c) into c from dual;
End;

```

Try calling in select statement

ORA-06572: Function TEST1 has out arguments
ORA-06512: at "SYS.DBMS_SQL", line 1721

error coming

How to call such function, same like we handle DML in function:

Declare

```

a number := 10;
b number := 5;

```

```

c number;
d number;
Begin
  d:= test1(10,5,c);
  dbms_output.put_line('out parameter value'||c);
  dbms_output.put_line('return parameter value'||d);
End;

```

```

1  Declare
2
3  a number := 10;
4  b number := 5;
5  c number;
6  d number;
7  Begin
8    d:= test1(10,5,c);
9    dbms_output.put_line('out parameter value'||c);
10   dbms_output.put_line('return parameter value'||d);
11
12 End;|

```

Statement processed.
out parameter value15
return parameter value5

Handwritten notes:
- Red arrow from 'out parameter value15' to 'out' (above the arrow)
- Blue arrow from 'return parameter value5' to 'Return' (below the arrow)
- Red text: '→ Both are different'

```

create or replace function test1(A IN number, b in number, c out number, e out number) return number as
Begin
  c := a+b;
  e := a* b;
  return c;
End test1;

```

Function created.

Handwritten notes:
- Purple text: '2 out parameters' with an arrow pointing to 'c out number, e out number'
- Green text: 'Returning the out Parameter itself' with an arrow pointing to 'return c;'
- Green text: 'allowed' with a checkmark next to 'Function created.'

Anything we can do, but just remember one thing that function will return 1 value. Out parameter will be used, suppose this function used in a procedure and I want to use that out value there then we can do.

Same with Proc as OUT parameter:

```
CREATE OR REPLACE PROCEDURE PROC1(p_emp_id IN NUMBER, amnt OUT NUMBER)
AS BEGIN
SELECT salary INTO amnt FROM hr.employees WHERE employee_id = p_emp_id;
END;
```

Call the proc

```
DECLARE
amount NUMBER;
BEGIN
PROC1(101, amount);
dbms_output.put_line(amount);
END;
```

```
DECLARE
amount NUMBER;
BEGIN
PROC1(101, amount);
dbms_output.put_line(amount);
END;
```

out parameter

```
Statement processed.
17000
```

IN OUT

- This parameter is used for both giving input and for getting output from the subprograms.
- It is a read-write variable inside the subprograms. Their values can be changed inside the subprograms.
- In the calling statement, these parameters should always be a variable to hold the value from the subprograms.

```
CREATE OR REPLACE PROCEDURE testinoutproc(testmessage IN OUT VARCHAR2)
IS
BEGIN
```

```
testmessage := 'testing my INOUT parameter' || testmessage;
```

```
END;
```

```

declare
    testmessage varchar2(100) := 'vishal jaiswal classes';
Begin
    testinginoutproc(testmessage);
    dbms_output.put_line(testmessage);
End;

```

```

Statement processed.
tetsing my inout parametervishal jaiswal classes

```

out *in*

```

CREATE OR REPLACE PROCEDURE update_employee_salary (
    emp_id IN NUMBER,
    new_salary IN OUT NUMBER
) AS
BEGIN
    new_salary := new_salary + 1000; -- Increase the salary by 1000
END;

```

In this example, new_salary is defined as an INOUT parameter. It can be both an input (IN) and output (OUT) parameter.

```

9 v DECLARE
10     emp_id NUMBER := 101;
11     salary NUMBER := 50000;
12 v BEGIN
13     -- Call the procedure with INOUT parameter
14     update_employee_salary(emp_id, salary);
15     DBMS_OUTPUT.PUT_LINE('New Salary: ' || salary);
16 END;
17

```

inout parameter for our use

```

Statement processed.
New Salary: 51000

```

In this example, emp_id is an input parameter, and salary is an INOUT parameter. After calling the procedure, the value of salary is updated by the procedure.

We can use the same functionality in function as well. remember, concept is same:

```
CREATE OR REPLACE FUNCTION calculate_tax (  
    income IN NUMBER,  
    tax_rate IN OUT NUMBER  
) RETURN NUMBER AS  
    tax NUMBER;  
BEGIN  
    tax := income * tax_rate;  
    tax_rate := tax_rate * 0.01; -- Convert tax_rate to a percentage  
    RETURN tax;  
END;
```

INOUT parameters are useful when you need to pass data into a procedure or function for processing and then receive modified or calculated data back.